

IoT (Internet of Things)

Question 1

1. Create Delay Routine for Buzzing LED

To create a delay routine for a buzzing LED, you can follow these steps:

Hardware Setup

1. Connect an LED to one of the digital pins on your Arduino board (e.g., pin 13).
2. Connect a buzzer or a piezo element to another digital pin (e.g., pin 8).

Software Implementation

3. Open the Arduino IDE and create a new sketch.
4. Add the following code:

C++

```
int ledPin = 13; // LED pin
int buzzerPin = 8; // Buzzer pin

void setup() {
  pinMode(ledPin, OUTPUT); // Set LED pin as output
  pinMode(buzzerPin, OUTPUT); // Set buzzer pin as output
}

void loop() {
  digitalWrite(ledPin, HIGH); // Turn LED on
  tone(buzzerPin, 2000); // Start buzzing
  delay(500); // Wait for 500 milliseconds
  digitalWrite(ledPin, LOW); // Turn LED off
  noTone(buzzerPin); // Stop buzzing
  delay(500); // Wait for 500 milliseconds
}
```

This code will turn the LED on, start the buzzer, wait for 500 milliseconds, turn the LED off, stop the buzzer, and then wait for another 500 milliseconds before repeating the cycle.

2. Toggle LED(s) by Switching

To toggle LED(s) by switching, you can follow these steps:

Hardware Setup

5. Connect one or more LEDs to digital pins on your Arduino board (e.g., pins 2, 3, and 4).
6. Connect a switch or a button to another digital pin (e.g., pin 7).

Software Implementation

7. Open the Arduino IDE and create a new sketch.
8. Add the following code:

C++

```
int led1Pin = 2; // LED 1 pin
int led2Pin = 3; // LED 2 pin
int led3Pin = 4; // LED 3 pin
int switchPin = 7; // Switch pin

int ledState = LOW; // Initial LED state

void setup() {
  pinMode(led1Pin, OUTPUT); // Set LED 1 pin as output
  pinMode(led2Pin, OUTPUT); // Set LED 2 pin as output
  pinMode(led3Pin, OUTPUT); // Set LED 3 pin as output
  pinMode(switchPin, INPUT_PULLUP); // Set switch pin as input with
  internal pullup resistor
}

void loop() {
  if (digitalRead(switchPin) == LOW) { // Switch is pressed
    ledState = !ledState; // Toggle LED state
    digitalWrite(led1Pin, ledState); // Set LED 1 state
    digitalWrite(led2Pin, ledState); // Set LED 2 state
    digitalWrite(led3Pin, ledState); // Set LED 3 state
    delay(200); // Debounce delay
  }
}
```

This code will toggle the state of the three LEDs (on/off) every time the switch is pressed. The `delay(200)` function is used to debounce the switch, preventing multiple toggling from a single press.

Question 2

Hardware Setup

9. Connect the DHT11 sensor to the Arduino:

- DHT11 pin 1 (VCC) to Arduino 5V pin
- DHT11 pin 2 (DATA) to Arduino digital pin 2
- DHT11 pin 3 (NC) - not connected
- DHT11 pin 4 (GND) to Arduino GND pin

10. Connect the LED displays to the Arduino:

- Connect the data pins of the LED displays to Arduino digital pins 4, 5, 6, 7, 8, 9, 10, 11
- Connect the common cathode pin of the LED displays to Arduino GND

Software Implementation

11. Install the DHT sensor library in the Arduino IDE:

- Go to Sketch > Include Library > Manage Libraries
- Search for "DHT sensor" and install the "DHT sensor library by Adafruit"

12. Create a new sketch and add the following code:

C++

```
#include <DHT.h>
#include <DHT_U.h>

#define DHTPIN 2    // Digital pin connected to the DHT sensor
#define DHTTYPE DHT11 // DHT 11
DHT_Unified dht(DHTPIN, DHTTYPE);

int segmentPins[] = {4, 5, 6, 7, 8, 9, 10, 11}; // Pins connected to
LED display segments

void setup() {
  Serial.begin(9600);
  dht.begin();
  for (int i = 0; i < 8; i++) {
```

```

        pinMode(segmentPins[i], OUTPUT);
    }
}

void loop() {
    sensors_event_t event;
    dht.temperature().getEvent(&event);
    float temperature = event.temperature;
    dht.humidity().getEvent(&event);
    float humidity = event.relative_humidity;

    Serial.print("Temperature: ");
    Serial.print(temperature);
    Serial.println("°C");
    Serial.print("Humidity: ");
    Serial.print(humidity);
    Serial.println("%");

    displayTemperature(temperature);
    displayHumidity(humidity);
    delay(2000);
}

void displayTemperature(float temp) {
    // Code to display temperature on LED displays
}

void displayHumidity(float hum) {
    // Code to display humidity on LED displays
}

```

13. Implement the `displayTemperature()` and `displayHumidity()` functions to display the readings on the LED displays. You can use techniques like multiplexing or shift registers to control the LED displays.
14. Upload the code to the Arduino and open the Serial Monitor to see the temperature and humidity readings.

The DHT11 sensor measures temperature and humidity by detecting changes in electrical resistance between two electrodes. The sensor converts the resistance measurements to

digital values and transmits them to the Arduino through a single-wire serial interface. By interfacing the DHT11 sensor with the Arduino and using the DHT sensor library, you can easily obtain temperature and humidity readings. The LED displays can be used to visually present the sensor data.

Question 3

Hardware Setup

15. Connect the DHT11 sensor to the Arduino:

- DHT11 pin 1 (VCC) to Arduino 5V pin
- DHT11 pin 2 (DATA) to Arduino digital pin 2
- DHT11 pin 3 (NC) - not connected
- DHT11 pin 4 (GND) to Arduino GND pin

16. Connect an LED to an Arduino digital pin (e.g., pin 13 for the built-in LED).

Software Implementation

17. Include the necessary libraries:

C++

```
#include <DHT.h>
#include <DHT_U.h>
```

18. Define the DHT11 sensor and LED pins:

C++

```
#define DHTPIN 2    // Digital pin connected to the DHT sensor
#define DHTTYPE DHT11 // DHT 11
#define LED_PIN 13  // LED pin

DHT_Unified dht(DHTPIN, DHTTYPE);
```

19. Set the temperature and humidity thresholds:

C++

```
float temperatureThreshold = 25.0; // Temperature threshold in °C
float humidityThreshold = 70.0;    // Humidity threshold in %
```

20. In the `setup()` function, initialize the sensor and the LED pin:

```
C++
void setup() {
  Serial.begin(9600);
  dht.begin();
  pinMode(LED_PIN, OUTPUT);
}
```

21. In the `loop()` function, read the temperature and humidity values, and turn on the LED if either value exceeds the threshold:

```
C++
void loop() {
  sensors_event_t event;

  // Read temperature
  dht.temperature().getEvent(&event);
  float temperature = event.temperature;

  // Read humidity
  dht.humidity().getEvent(&event);
  float humidity = event.relative_humidity;

  // Print the readings
  Serial.print("Temperature: ");
  Serial.print(temperature);
  Serial.println("°C");
  Serial.print("Humidity: ");
  Serial.print(humidity);
  Serial.println("%");

  // Check if temperature or humidity is above the threshold
  if (temperature > temperatureThreshold || humidity >
humidityThreshold) {
    digitalWrite(LED_PIN, HIGH); // Turn on the LED
  } else {
```

```
    digitalWrite(LED_PIN, LOW); // Turn off the LED
  }

  delay(2000); // Wait for 2 seconds before the next reading
}
```

Question 4

Hardware Setup

- 22. Connect the NodeMCU board to your computer using a micro-USB cable.
- 23. Ensure you have the ESP8266 board package installed in the Arduino IDE.

Software Implementation

- 24. Include the necessary libraries:

```
C++
#include <ESP8266WiFi.h>
#include <ESP8266HTTPClient.h>
#include <WiFiClient.h>
```

- 25. Set up the WiFi connection:

```
C++
const char* ssid = "YOUR_SSID";
const char* password = "YOUR_PASSWORD";
```

- 26. Define the server URL and parameters for sending data:

```
C++
String serverName = "http://example.com/api/data";
String apiKey = "YOUR_API_KEY";
String sensorData = "sensor_value=123";
```

- 27. In the `setup()` function, connect to WiFi:

```
C++
```

```

void setup() {
  Serial.begin(115200);
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(1000);
    Serial.println("Connecting to WiFi...");
  }
  Serial.println("Connected to WiFi");
}

```

28. In the `loop()` function, send an HTTP GET request to the server:

C++

```

void loop() {
  if (WiFi.status() == WL_CONNECTED) {
    HTTPClient http;
    http.begin(serverName + "?api_key=" + apiKey + "&" + sensorData);
    int httpResponseCode = http.GET();
    if (httpResponseCode > 0) {
      String response = http.getString();
      Serial.println("HTTP Response code: " +
String(httpResponseCode));
      Serial.println("Response: " + response);
    } else {
      Serial.print("Error code: ");
      Serial.println(httpResponseCode);
    }
    http.end();
  } else {
    Serial.println("WiFi Disconnected");
  }
  delay(5000); // Wait for 5 seconds before sending the next request
}

```

29. To receive data from the server using HTTP GET requests, modify the server URL and parameters accordingly:

C++

```
String serverName = "http://example.com/api/get_data";  
String apiKey = "YOUR_API_KEY";
```

30. In the `loop()` function, send an HTTP GET request to the server to receive data:

```
c++  
void loop() {  
  if (WiFi.status() == WL_CONNECTED) {  
    HTTPClient http;  
    http.begin(serverName + "?api_key=" + apiKey);  
    int httpResponseCode = http.GET();  
    if (httpResponseCode > 0) {  
      String response = http.getString();  
      Serial.println("HTTP Response code: " +  
String(httpResponseCode));  
      Serial.println("Response: " + response);  
      // Parse the received data and perform necessary actions  
    } else {  
      Serial.print("Error code: ");  
      Serial.println(httpResponseCode);  
    }  
    http.end();  
  } else {  
    Serial.println("WiFi Disconnected");  
  }  
  delay(5000); // Wait for 5 seconds before sending the next request  
}
```

Remember to replace `YOUR_SSID`, `YOUR_PASSWORD`, `YOUR_API_KEY`, and the server URLs with your actual values. This code demonstrates the basic structure for sending and receiving data using HTTP GET requests with the NodeMCU board. You can modify the server URLs, parameters, and data parsing based on your specific requirements and the API documentation of the server you are communicating with.

Question 5 :

Hardware Setup

31. Connect the HC-05 Bluetooth module to the Arduino:

- HC-05 VCC pin to Arduino 5V
- HC-05 GND pin to Arduino GND
- HC-05 TX pin to Arduino RX pin (e.g., pin 0)
- HC-05 RX pin to Arduino TX pin (e.g., pin 1)

32. Connect a sensor, such as a DHT11 temperature and humidity sensor, to the Arduino.

Software Implementation

33. Install the necessary libraries:

- DHT sensor library: <https://github.com/adafruit/DHT-sensor-library>
- SoftwareSerial library (included in Arduino IDE)

34. Write the Arduino code to read the sensor data and send it to the Bluetooth module:

C++

```
#include <SoftwareSerial.h>
#include <DHT.h>

#define DHTPIN 2      // Digital pin connected to the DHT sensor
#define DHTTYPE DHT11 // DHT 11
DHT dht(DHTPIN, DHTTYPE);

SoftwareSerial btSerial(0, 1); // RX, TX

void setup() {
  Serial.begin(9600);
  btSerial.begin(9600);
  dht.begin();
}

void loop() {
  float temperature = dht.readTemperature();
  float humidity = dht.readHumidity();

  if (isnan(temperature) || isnan(humidity)) {
    Serial.println("Failed to read from DHT sensor!");
  }
}
```

```

    return;
}

Serial.print("Temperature: ");
Serial.print(temperature);
Serial.println("°C");
Serial.print("Humidity: ");
Serial.print(humidity);
Serial.println("%");

btSerial.print("Temperature: ");
btSerial.print(temperature);
btSerial.println("°C");
btSerial.print("Humidity: ");
btSerial.print(humidity);
btSerial.println("%");

delay(2000);
}

```

35. On the smartphone side, you'll need to develop an app to receive the Bluetooth data. You can use a pre-existing app like "Serial Bluetooth Terminal" or create your own app using a framework like MIT App Inventor or Android Studio.

- In the app, you'll need to pair the smartphone with the HC-05 Bluetooth module and establish a connection.
- Once connected, the app should be able to receive the sensor data sent from the Arduino over Bluetooth.

The key points are:

36. Connect the HC-05 Bluetooth module to the Arduino using the appropriate pins.
37. Read the sensor data (in this case, temperature and humidity) using the DHT sensor library.
38. Send the sensor data to the Bluetooth module using the SoftwareSerial library.
39. Develop a smartphone app to receive and display the Bluetooth data.

Remember to adjust the pin numbers and sensor type according to your specific hardware setup.

Question 6

Hardware Setup

40. Connect a temperature sensor, such as a DHT11 or DHT22, to your microcontroller board (e.g., Arduino, NodeMCU, etc.).

- Connect the VCC pin of the DHT sensor to the 3.3V or 5V pin of the microcontroller.
- Connect the GND pin of the DHT sensor to the GND pin of the microcontroller.
- Connect the DATA pin of the DHT sensor to a digital pin on the microcontroller.

41. Ensure you have a microcontroller board with Wi-Fi or Ethernet connectivity, such as an Arduino with a Wi-Fi shield or a NodeMCU.

Software Implementation

42. Set up the ThingSpeak account and channel:

- Create a ThingSpeak account at <https://thingspeak.com>.
- Create a new channel and note down the "Channel ID" and "Write API Key".

43. Install the necessary libraries:

- For Arduino, install the "ThingSpeak" library from the Arduino Library Manager.
- For NodeMCU, you can use the "ESP8266WiFi" and "ThingSpeak" libraries.

44. Write the code to read the temperature and send it to ThingSpeak:

C++

```
#include <ThingSpeak.h>
#include <DHT.h>

// Replace with your channel details
unsigned long channelID = YOUR_CHANNEL_ID;
const char* writeAPIKey = "YOUR_WRITE_API_KEY";

// DHT sensor configuration
#define DHTPIN 2      // Digital pin connected to the DHT sensor
#define DHTTYPE DHT11 // DHT 11
DHT dht(DHTPIN, DHTTYPE);
```

```

void setup() {
  Serial.begin(9600);
  dht.begin();

  // Initialize ThingSpeak
  ThingSpeak.begin(client);
}

void loop() {
  // Read temperature from the DHT sensor
  float temperature = dht.readTemperature();

  if (isnan(temperature)) {
    Serial.println("Failed to read from DHT sensor!");
    return;
  }

  Serial.print("Temperature: ");
  Serial.print(temperature);
  Serial.println("°C");

  // Send the temperature value to ThingSpeak
  ThingSpeak.setField(1, temperature);
  ThingSpeak.writeFields(channelID, writeAPIKey);

  delay(20000); // Wait for 20 seconds before the next reading
}

```

45. Replace the placeholders with your actual ThingSpeak channel ID and write API key.
46. Upload the code to your microcontroller board.

The code will read the temperature from the DHT sensor, print it to the serial monitor, and then send the temperature value to the ThingSpeak cloud platform. The data will be updated every 20 seconds. You can then view the temperature data on your ThingSpeak channel and use the various visualization and analysis tools provided by the platform.